# TBO Helm

## Phase 5: System Architecture and Implementation Plan

Prepared 9 November 2026 · Basis: Approach B (Phase 4) · Updated for: team of 2, consumer GPU, free-tier hosting

**Confirmed inputs:** team of two; a consumer laptop GPU [A: 4–6 GB VRAM]; free-tier hosting for an always-on demo. These are folded into every section below.

**Verification notes:** llama.cpp's llama-server exposes OpenAI-compatible POST /v1/chat/completions, /v1/completions, /v1/embeddings, GET /v1/models and /health, accepts the OpenAI tool-calling shape, JSON mode and json_schema grammar constraints, and cannot serve an embedding model and a chat model in one process (SetFit embeds in-process, so this is fine). The TBO mock API contract is unknown (U1); the adapter interface below is ours.

## A. Architecture summary

**System overview.** Helm is one Python service (helm-core) backed by one PostgreSQL database, one React front end (helm-ui), one local inference process (llama-server), and one batch evaluation package (helm-eval). Everything that changes the world outside Helm (mock IT/Admin/Finance calls, outbound messages, state transitions that trigger notifications) goes through a single function, the Governed Effect Boundary (GEB).

| Component | Responsibility |
|---|---|
| API layer | REST endpoints, auth, RBAC, input validation |
| Case engine | Cases, links, status machine, timeline, threads |
| Policy engine | Loads versioned, effective-dated YAML rule sets; evaluates routing, SLA, reserved-decision and eligibility rules; stamps rule versions |
| Clock engine | Due times from priority, check-in/LCD, statutory rules and business calendars; at-risk and breached flags |
| GEB | The only path for side effects: policy → reserved check → approval binding → idempotency ledger → adapter → audit |
| Workflow runner | Executes YAML workflow definitions step by step on a Postgres-backed queue; retries; parks exceptions; honours kill switches |
| Skill registry | Maps step types (call_adapter, classify, ask_llm, wait_approval, set_status, …) to implementations; new skills register without touching the runner |
| Model gateway | One client for the local OpenAI-compatible endpoint; Presidio masking; prompt versioning; token and time metering; provider switch by config |
| Intent and themes | SetFit classifier (in-process); seeded theme tagger; calibrated deferral to the LLM |
| Metrics and export | Automation rate, cost per workflow, cycle time, escalation rate; streaming CSV/JSON exports |
| helm-eval | Scenario runner, fault injector, injection corpus, simulated requester, baselines B0/B1, reports |

**Data flow.** Inputs arrive from the UI, the chat endpoint, seed loaders or scenario runners. They create or update cases and enqueue workflow steps. Steps read case state and rules, optionally call the model gateway for typed outputs (intent, theme, extracted fields), and request effects

through the GEB. The GEB writes approvals, ledger entries, audit rows and cost rows. Metrics and exports read from those tables.

**Control flow.** Deterministic. The workflow runner picks the next step from a definition; rules decide branching; the LLM never chooses which tool to call. Model outputs are typed values validated by schema before rules consume them.

**Model flow.** Text → Presidio mask → SetFit probability vector → if max probability ≥ calibrated threshold, accept; else → LLM (JSON-schema-constrained) → validate → if still low confidence, flag “needs human” (exception queue). Chat follows the same gateway with topic rails implemented as a rule set.

**External dependencies.** TBO mock APIs (HTTP, contract TBD); llama-server (local); PostgreSQL (local or managed free tier); Hugging Face model downloads at setup time only.

## Trust boundaries and security-sensitive components

| Boundary | Sensitive component | Control |
|---|---|---|
| Browser → API | Authentication, RBAC | Demo auth with signed session tokens; role checks on every endpoint |
| Untrusted text (agency emails, supplier replies, chat) → model | Model gateway | Masking; schema-constrained outputs; outputs can never be tool calls |
| Service → external effects | GEB | Deny by default; reserved catalogue; intent-bound approvals; idempotency |
| Service → model endpoint | Gateway | Local only by default; provider change requires config and is logged |
| Operator → rules | Policy engine | Rule sets are versioned files with effective dates and tests; HR edits through forms that write new versions, never in-place edits |
| Data at rest | Postgres | Synthetic only; retention jobs; no secrets in the DB |

# B. Diagrams

## High-level architecture

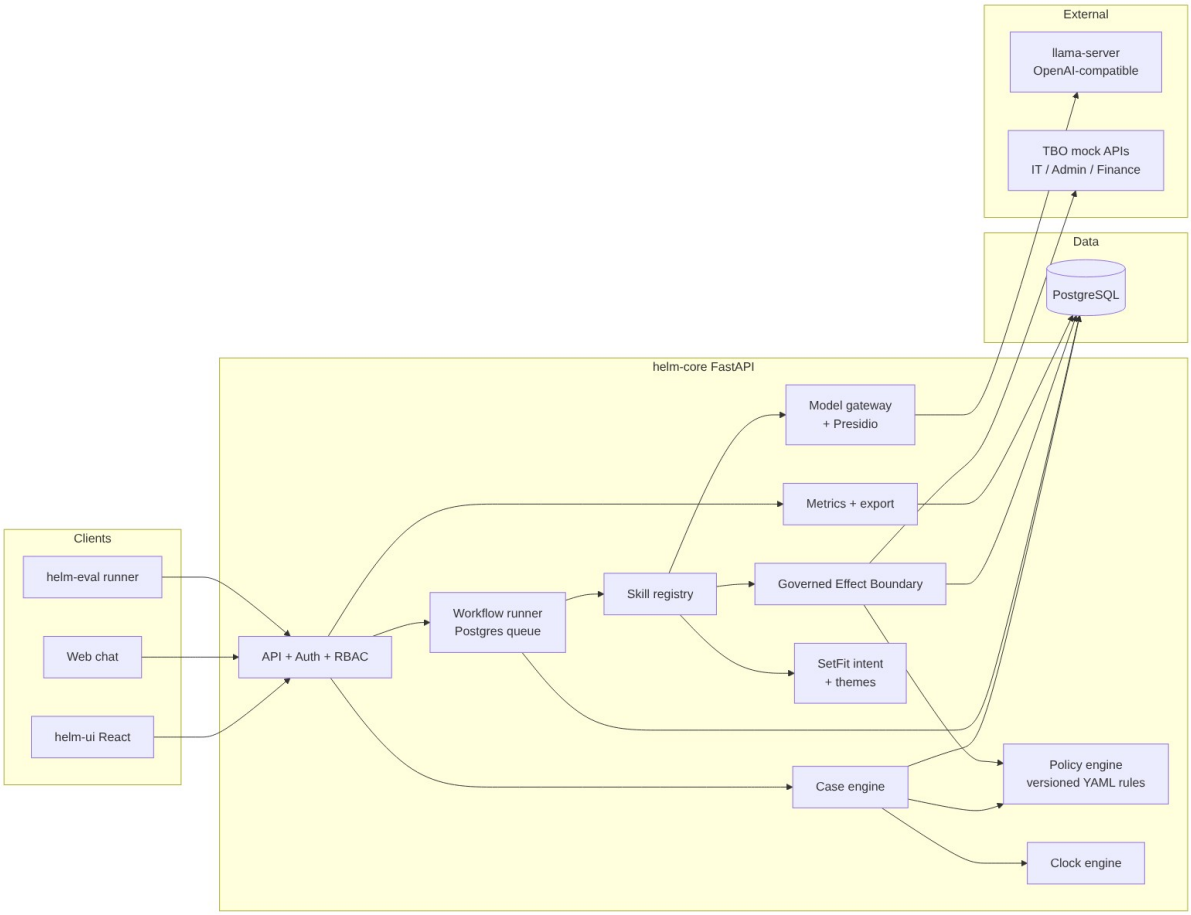



*Figure 1. High-level architecture: all side effects pass through the Governed Effect Boundary.*

## End-to-end onboarding workflow (sequence)

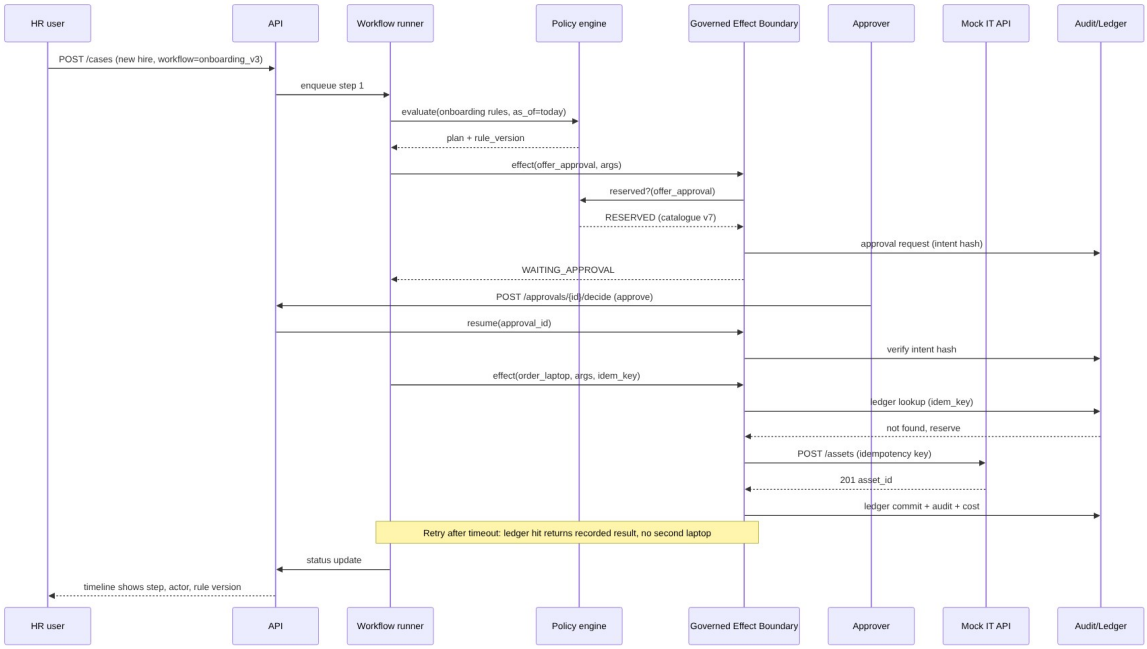


*Figure 2. Onboarding: reserved decision waits for a named approver; a retry hits the ledger and never orders a second laptop.*

## Deployment (free-tier hosted instance plus laptop for live LLM demo)

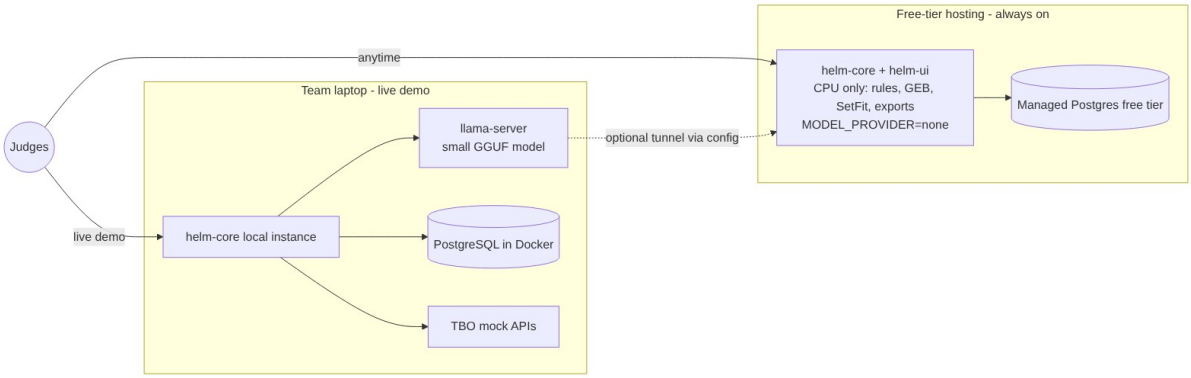



*Figure 3. Deployment: always-on CPU-only hosted instance; LLM-backed features run from the team laptop.*

## C. Component specification

| Component | Purpose | Technology | Alternatives considered | Input → Output | Interfaces | Storage | Failure modes | Monitoring | Licence / cost |
|---|---|---|---|---|---|---|---|---|---|
| API layer | HTTP surface, auth, validation | FastAPI + Pydantic v2 | Django REST (heavier), Flask (less validation) | HTTP JSON → domain calls | REST /api/v1/*, OpenAPI | None | Validation and auth failures | Structured request logs with request_id | MIT / free |
| Case engine | Cases, links, status machine, timeline, threads | Python + SQLAlchemy 2 | Reusing Zammad/Frappe models (AGPL, rejected) | Commands → case state + events | Python module API | cases, case_links, threads, thread_recipients, timeline_events | Invalid transitions; link cycles | Event counts per type | MIT / free |
| Policy engine | Evaluate versioned, effective-dated rules | Python evaluator over YAML (JSON-Schema validated) | OPA/Rego (deferred); json-rules-engine | Facts + rule set + as-of date → decision + rule_version + reason | evaluate(ruleset, facts, as_of) | rule_sets (immutable versioned rows) | Ambiguous rules; missing effective version | Decision log with rule_version; coverage tests | MIT / free |
| Clock engine | SLA and statutory deadlines | Python + workalendar or in-house calendars | Celery beat (overkill) | Case facts + rule version → due_at, risk state | compute_due(case), tick() | sla_clocks, calendars | Timezone bugs; holiday gaps | At-risk / breach counters | MIT / free (workalendar MIT, NV) |
| GEB | Sole path for side effects | Python module, one entry point | Cerberus / agent-ledger (reviewed; not adopted to keep one codebase) | EffectRequest → EffectResult / WAITING_APPROVAL / DENIED | effect(principal, type, args, scope) | approvals, effect_ledger (unique idem_key), audit_events, cost_events | Ledger race (unique constraint + ON CONFLICT); adapter timeout | Every call logged; denial and duplicate counters | MIT / free |
| Workflow runner | Execute YAML workflows | Python worker; Postgres queue (SELECT … FOR UPDATE SKIP LOCKED) | Temporal (deferred), Celery + Redis | Workflow def + case → steps executed | enqueue(step), run_once() | workflow_runs, steps | Stuck steps; poison messages | Queue depth, step latency, retries | MIT / free |
| Skill registry | Pluggable step implementations | Decorator-based registry | pluggy | Step type → callable | @skill("name") | None | Missing skill for a step type | Registry dump at startup | MIT / free |

| Component | Purpose | Technology | Alternatives considered | Input → Output | Interfaces | Storage | Failure modes | Monitoring | Licence / cost |
|---|---|---|---|---|---|---|---|---|---|
| Model gateway | Local LLM access, masking, metering | httpx client to OpenAI-compatible endpoint; Presidio | LiteLLM SDK (avoided to minimise deps; enterprise/ restrictions) | Prompt (versioned) + schema → validated JSON + usage | complete(prompt_id, vars, schema) | model_calls (tokens, latency, model, prompt_version, masked flag) | Endpoint down → park; schema violation → retry once then park | Tokens, latency, escalation rate | MIT / Apache-2.0 / free |
| Intent & themes | Intent suggestion, theme tags | SetFit + sentence-transformer body; isotonic calibration; seeded theme rules | Keyword triggers (B0); BERTopic (deferred) | Text → label, probability, threshold decision | classify(text) | intent_labels, calibration | Drift; low-data classes | Held-out accuracy; deferral rate | Apache-2.0 (SetFit NV); body licence NV |
| Metrics & export | Rubric metrics, uncapped export | SQL views + streaming responses | Separate BI tool | Tables → JSON/CSV streams | /metrics, /export | Views only | Long exports (mitigated by streaming) | Export sizes and durations | MIT / free |
| helm-ui | Agent, HR, approver, lead screens; chat | React + Vite + Tailwind | Next.js (SSR unnecessary) | API JSON → screens | REST client | Browser memory | API errors; empty states | Client error logging | MIT / free |
| helm-eval | Scenarios, faults, baselines, reports | Python package; pytest; pandas | Reusing τ²-bench harness (pattern only) | Scenario YAML → metrics JSON + HTML report | CLI helm-eval run --suite X --system helm\|b0\|b1 | eval_runs or files | Non-determinism in LLM baseline (seeds, k repeats) | Per-run manifest with model hashes | MIT / free |
| Mock APIs | TBO IT/Admin/Finance | Provided by TBO (unknown) | Own stubs for early development | HTTP | Adapter per API | Adapter-side ledger | Unknown idempotency behaviour (U1) | Adapter call logs | Per TBO |

## D. Technology stack

No paid dependencies. The only optional paid item is a hosted model as fallback if VRAM is insufficient; it stays config-switchable and metered.

| Technology | Role | Why selected | Category | Licence / restrictions | Hardware / ops | Integration | Fallback |
|---|---|---|---|---|---|---|---|
| Python 3.12 | Core language | ML tooling, team familiarity | Open-source | PSF | Any | Low | — |

| Technology | Role | Why selected | Category | Licence / restrictions | Hardware / ops | Integration | Fallback |
|---|---|---|---|---|---|---|---|
| FastAPI + Pydantic v2 + Uvicorn | API, validation | Typed schemas, auto OpenAPI, async | Open-source | MIT | Any | Low | Flask |
| SQLAlchemy 2 + Alembic | ORM, migrations | Mature; explicit SQL when needed | Open-source | MIT | Any | Low | Raw psycopg |
| PostgreSQL 16 | Storage, queue, audit | Transactions, SKIP LOCKED queues, JSONB, cursors for streaming | Open-source | PostgreSQL Licence | Laptop container or free managed tier (limits to verify) | Low | SQLite for dev only |
| React 18 + Vite + Tailwind | UI | Fast dev, no SSR needed | Open-source | MIT | Any | Low–medium | Plain HTML templates |
| llama.cpp llama-server | Local inference | OpenAI-compatible endpoints, JSON-schema grammar, GPU offload, GGUF | Open-source | MIT (NV) | Consumer GPU | Low | Ollama; hosted Apache-2.0 model via same client |
| Qwen3-1.7B / Gemma 4 E2B (4 GB) or Qwen3-4B (6 GB) | Default LLM | Apache-2.0 open weights; fits a consumer GPU | Open-weight | Apache-2.0 (Qwen3 official; Gemma 4 per secondary source, verify model card) | 4–6 GB VRAM | Low | Smaller quant; hosted model |
| SetFit + sentence-transformers | Intent classifier | Few-shot, CPU-friendly, deterministic probabilities | Open-source (+ open-weight body) | Apache-2.0 (NV); body licence NV | CPU | Low | Logistic regression on embeddings |
| scikit-learn | Calibration, baselines | Isotonic regression, metrics | Open-source | BSD-3 | CPU | Low | Manual threshold sweep |
| Microsoft Presidio | PII masking | Pluggable recognisers for Indian identifiers | Open-source | MIT | CPU; spaCy model download | Medium | Regex-only masker |
| PyYAML + jsonschema | Rules and workflow definitions | Human-editable, schema-validated | Open-source | MIT | Any | Low | JSON files |
| workalendar | Business calendars | Regional holiday calendars | Open-source | MIT (NV) | Any | Low | Hand-maintained tables |
| httpx | HTTP client | Async, timeouts, retries | Open-source | BSD-3 | Any | Low | requests |
| pytest + Hypothesis | Tests, property tests for idempotency | Standard | Open-source | MIT / MPL-2.0 | Any | Low | pytest only |

| Technology | Role | Why selected | Category | Licence / restrictions | Hardware / ops | Integration | Fallback |
|---|---|---|---|---|---|---|---|
| Docker Compose | Local deployment | One command for judges | Open-source | Apache-2.0 | Any | Low | Manual processes |
| LangGraph (B1 baseline only) | ReAct agent baseline | Representative LLM-only agent | Open-source | MIT | Any | Low | Hand-written ReAct loop |

## E. Data and model pipeline

| Asset | Acquisition | Validation | Annotation | Split | Versioning |
|---|---|---|---|---|---|
| TBO synthetic HR dataset | Starter kit | Schema check; referential integrity | None (provided) | n/a | Read-only fixture with hash |
| OpsDesk seed | scripts/seed_opsdesk.py with Faker (licence NV) and templates; fixed random seed | TBOH uniqueness; date consistency | Intent labels at generation, then manually reviewed by the non-author teammate (~300 items) | 70/15/15 stratified by intent, split by TBOH | Seed file + generator hash committed |
| Intent paraphrase augmentation | LLM paraphrases via local Apache-2.0 model | Dedup; 20% reviewer spot-check | Inherit label | Train only | Prompt version recorded |
| PII test strings | Synthetic Aadhaar-like, PAN-like, phone and email patterns (format-valid, non-real) | Format validators | Spans known by construction | Test only | Committed |
| Scenario suite (30) | Authored YAML | Schema; dry-run against stub adapters | Expected outcomes encoded | n/a | Suite version in reports |
| Injection corpus (30–50) | Authored payloads in realistic agency/supplier text | Each has an expected “no action” outcome | By construction | Test only | Committed |

- **Leakage prevention:** split by TBOH and agency; calibration set carved from validation, never test; scenario expectations never shown to the LLM baseline.
- **Preprocessing:** normalise whitespace, strip signatures and quoted replies, mask PII before any embedding or LLM call; keep raw (synthetic) text for audit alongside the masked variant.
- **Training / inference:** SetFit contrastive fine-tune on train (CPU); isotonic calibration on validation; threshold τ chosen by cost-minimisation subject to an accuracy target; LLM inference only, JSON-schema constrained, temperature 0, prompts versioned.
- **Evaluation:** intent top-1/top-3 and macro-F1 vs keyword baseline; deferral rate at τ; system metrics from helm-eval (violations, duplicates, unauthorised access, pass^k, CuP-style completion, deadline correctness, injection success, INR per workflow).
- **Reproducibility:** model files pinned by SHA-256; pinned dependencies with hashes; every eval run writes manifest.json (commit, suite version, model hashes, seeds, τ, hardware, GPU amortisation parameters); rules and workflows are immutable versioned rows.

## F. API and interface specification

All endpoints under /api/v1, JSON, bearer session token; errors use RFC 9457 problem details.

| Method | Path | Purpose | Roles |
|---|---|---|---|
| POST | /auth/login | Demo login → session token | all |
| GET/POST | /cases | List (role-scoped) / create | agency (own), agent, lead |

| Method | Path | Purpose | Roles |
|---|---|---|---|
| GET | /cases/{id} | Case with links, timeline, threads, SLA state | scoped |
| POST | /cases/{id}/transition | Status change via GEB | agent, EM |
| POST | /cases/{id}/assign | Assign / reassign / unassign via GEB | agent, lead |
| POST | /cases/{id}/escalate | First-class escalation via GEB | agent |
| GET/ POST | /cases/{id}/threads | Threads with explicit recipients | agent |
| POST | /threads/{id}/messages | Internal note or external reply (external via GEB) | agent |
| GET | /queues/{view} | triage, mine, region, intent, linked, escalation | agent, EM, lead |
| GET/ POST | /approvals, /approvals/{id}/decide | Reserved-decision approvals | named approvers |
| GET | /exceptions | Human exception queue | HR ops, lead |
| GET/ POST | /rules/{set} | List versions / create new version with effective date | HR ops, lead |
| POST | /controls/pause | Kill switch for workflow or agent | HR ops, lead |
| POST | /chat | Conversational access (typed intents → actions via GEB) | employee, manager |
| GET | /insights/themes?week= | Top themes with sample cases | lead, product |
| GET | /metrics/summary | Automation, cost, cycle time, escalation | lead |
| GET | /export/{table}?format=csv | Streaming export, cursor-paginated | lead |

## Sample: create case

```
POST /api/v1/cases
{
  "title": "Refund not received for cancelled booking",
  "description": "We cancelled TBOH-88213 before LCD, refund still pending...",
  "agency_id": "AG-014", "tboh": "TBOH-88213", "priority": "high",
  "check_in": "2026-11-12", "lcd": "2026-11-10T18:00:00+05:30", "region": "IN-WEST"
}

200 OK
{
  "id": "CASE-10421", "status": "new",
  "intent": {"suggested": "refund_delay", "confidence": 0.91, "source": "setfit", "thresh
old": 0.78},
  "links": [{"case_id": "CASE-09877", "reason": "same TBOH"},
            {"case_id": "CASE-10102", "reason": "same TBOH + intent"}],
  "routing": {"owner": "agent-07", "reason": "region=IN-WEST, priority=high, available",
              "rule_version": "routing@v4"},
  "sla": {"due_at": "2026-11-10T12:00:00+05:30", "state": "at_risk", "rule_version": "sla
@v3"}
}
```

## Sample: GEB effect (internal interface)

```
EffectRequest(
    principal=Principal(user_id="agent:onboarding", on_behalf_of="hr-ops-02", roles=["hr_
ops"]),
    effect_type="it.order_laptop",
    args={"employee_id": "E-2031", "model": "standard-14"},
    scope=Scope(workflow_run_id="WR-331", step_id="order_laptop"),
)
# -> EffectResult(status="committed", external_ref="ASSET-5512",
#                 idem_key="WR-331:it.order_laptop:3f9c...", audit_id="AUD-77120",
#                 cost_inr=0.0, rule_version="reserved@v7")
```

## Sample: reserved-decision rule set (versioned YAML)

```
ruleset: reserved_decisions
version: 7
effective_from: 2026-11-01
reserved:
  - effect: hr.accept_resignation
    approver_role: hr_manager
  - effect: hr.approve_offer
    approver_role: hiring_manager
  - effect: payroll.sign_off_run
    approver_role: finance_controller
  - effect: ops.approve_refund
    when: args.amount_inr > 50000
    approver_role: escalation_manager
route_only:
  - effect: hr.posh_complaint
    route_to: internal_committee
```

## Sample: statutory clock rule set

```
ruleset: statutory_clocks
version: 2
effective_from: 2025-11-21
clocks:
  - id: final_settlement
    trigger: hr.exit_effective
    due: "+2 business_days"
    calendar: IN-national
    source: "Code on Wages 2019, in force 21 Nov 2025 (verify against TBO policy)"
  - id: dpdp_grievance
    trigger: dpdp.request_received
    due: "+90 calendar_days"
    source: "DPDP Rules 2025 (verify)"
```

## Sample: workflow definition (excerpt)

```
workflow: onboarding
version: 3
steps:
  - id: approve_offer
    skill: effect
    effect: hr.approve_offer        # reserved -> waits for approver
  - id: order_laptop
```

```
    skill: effect
    effect: it.order_laptop
    args: {employee_id: "{{employee.id}}", model: "{{policy.laptop_model}}"}
    retry: {max: 3, backoff: exponential}
  - id: access_card
    skill: effect
    effect: admin.issue_card
  - id: payroll_draft
    skill: effect
    effect: finance.create_payroll_profile
  - id: payroll_signoff
    skill: effect
    effect: payroll.sign_off_run    # reserved
  - id: notify_manager
    skill: effect
    effect: notify.message
    recipients: ["{{employee.manager_id}}"]
```

**Internal events (timeline_events / audit_events):** case.created, case.linked, case.routed, case.status_changed, case.escalated, thread.message_sent, effect.requested, effect.waiting_approval, effect.committed, effect.denied, effect.replayed, approval.decided, workflow.parked, control.paused, model.called.

## G. Security, privacy and reliability

| Concern | MVP measure | Deferred |
|---|---|---|
| Authentication | Demo users with bcrypt-hashed passwords; signed, expiring session tokens; documented accounts | SSO / IdP |
| Authorisation | Role → permission map; row-level agency filter in every case query; GEB re-checks the acting principal's permissions | Attribute-based policies in OPA |
| Reserved decisions | Catalogue in versioned rules; GEB denies agent completion; approval must match intent hash; requester ≠ approver | Multi-step approvals |
| Secrets | .env only, never committed; none in rules or DB; secret scan in CI | Vault |
| Input validation | Pydantic on every endpoint; JSON Schema on rules and workflows; size limits | — |
| Untrusted text | Masked before model calls; outputs schema-validated; outputs never map to effects directly | CaMeL-style capabilities |
| Rate limits | Per-session token bucket on /chat and /export | Gateway-level limits |
| Error handling | Steps park to the exception queue with context; adapter timeouts and bounded retries; ledger guards retries | Dead-letter analytics |
| Idempotency | Unique idem_key per (workflow_run, effect_type, args_hash); reserve-then-commit with ON CONFLICT; key passed downstream when supported (U1) | Semantic dedup |
| Kill switch | controls table checked before every step; UI toggle | Per-tenant controls |
| Audit | Append-only audit_events with actor, on_behalf_of, inputs, rule_version, prompt_version, model, reasoning, cost; DB role cannot UPDATE/DELETE | WORM storage |

| Concern | MVP measure | Deferred |
|---|---|---|
| Data protection (DPDP principles) | Purpose tag per field; retention job; access/correction requests as a case type; synthetic data only | Consent manager integration |
| Backups / recovery | pg_dump nightly in demo; Compose volume; restore runbook tested once | PITR |
| Observability | Structured JSON logs with request_id and workflow_run_id; /health; counters for denials, duplicates, parks | OpenTelemetry |
| Hosted instance | MODEL_PROVIDER=none: LLM steps park to the human queue (required O6 behaviour); README warns of free-tier cold starts | Tunnel bridge to laptop model, by config only |

## H. Repository structure

```
helm/
├── README.md                # setup, demo accounts, checklist (P1–6, O1–8), assumptions
, licences
├── docker-compose.yml       # core, ui, postgres, mock-apis
├── .env.example
├── core/                    # helm-core (Python)
│   ├── helm/
│   │   ├── api/             # FastAPI routers, auth, RBAC dependencies
│   │   ├── cases/           # case engine: models, status machine, links, threads, time
line
│   │   ├── policy/          # rule loader, evaluator, versioning, effective dates
│   │   ├── clocks/          # SLA + statutory clocks, calendars
│   │   ├── geb/             # Governed Effect Boundary: approvals, ledger, audit, cost
│   │   ├── workflows/       # runner, queue, step model, parking, kill switch
│   │   ├── skills/          # registry + built-in skills
│   │   ├── gateway/         # model client, masking (Presidio), prompt registry, meteri
ng
│   │   ├── nlu/             # SetFit training/inference, calibration, theme tagger
│   │   ├── adapters/        # mock IT/Admin/Finance clients behind a common interface
│   │   ├── metrics/         # SQL views, summaries, streaming export
│   │   └── db/              # SQLAlchemy models, Alembic migrations
│   ├── tests/               # unit + property tests (idempotency, RBAC, reserved, clock
s)
│   └── pyproject.toml
├── ui/                      # helm-ui (React + Vite + Tailwind)
├── definitions/             # everything a service line needs, no core code
│   ├── opsdesk/             # intents, sla, routing, themes, seed/
│   ├── hr_onboarding/       # workflow, reserved, statutory_clocks, policy
│   └── _template/           # scaffold for the live extension test
├── prompts/                 # versioned prompt files
├── eval/                    # helm-eval: scenarios/, injections/, baselines/, runner/,
metrics/, reports/
├── scripts/                 # seed generators, model download with hash check, cost cal
culator
├── docs/                    # architecture, developer_guide, cost_method, assumptions,
demo_script
└── LICENSES/                # third-party licence inventory
```

**Why this layout:** the live extension test is won or lost in definitions/. A new service line is a new folder there plus, at most, a new skill in core/helm/skills/. Any other change to core/ during the test is a failure signal we measure with git diff --stat core/.

## I. Milestones and team allocation (team of two)

Effort in person-days (pd) is engineering judgement. About 56 pd total against roughly 50–56 available across ~5–6 weeks; the cut list below absorbs slippage.

| Milestone | Tasks | Depends on | Deliverables | P1 / P2 (pd) | Completion criteria | Likely blockers |
|---|---|---|---|---|---|---|
| M0 Setup & PoC (Round 1 weeks) | Repo scaffold; Compose with Postgres; llama-server with a small GGUF and JSON-schema output; SetFit smoke test; stub mock APIs; provision free-tier Postgres + app host and verify limits; confirm VRAM; architecture doc + 5-min video | — | Skeleton deployed to free tier; Round 1 document and video | 5 / 5 | Hello-world workflow executes one stub effect through a minimal GEB; model returns valid JSON | VRAM; starter kit not yet released; free-tier limits |
| M1 Core MVP-0 (build wk 1–2) | Case engine; policy engine + schemas; clock engine; GEB with ledger, approvals, audit, cost; runner + queue; RBAC; kill switch; adapters to real mocks; SetFit v1; queue views, case detail, approvals UI; onboarding workflow v1; 10 scenarios | M0; starter kit (U1, U2) | MVP-0 | 11 / 10 | Violations 0, duplicates 0, unauthorised 0; journeys A, B, D; onboarding happy path + one reserved + one fault | Mock API semantics; rule schema churn |
| M2 Integration (wk 3) | Side conversations; escalation queue; Insights (seeded themes); streaming export ≥50K; F&F and DPDP clocks; chat Q&A-only; dashboard; calibrated deferral; B0 and B1 baselines; 30 scenarios; injection corpus | M1 | Submission-scope features; eval report v1 | 6 / 7 | Journeys A–E pass; report with Helm vs B0 vs B1 | Chat quality on a small model; export performance |
| M3 Eval & hardening (wk 4a) | Fault-injection matrix; pass^k runs (k=5); projected large-model cost (no B2); cost method finalised; 2 extension drills; Tier 3 licence audit; bug fixes | M2 | Eval report v2; developer guide; cost_method.md; licence inventory | 4 / 3 | Hard zeros hold under fault injection; drills with core diff 0; report reproducible from manifest | Flaky LLM baseline (seeds, k) |
| M4 Demo & deployment (wk 4b) | Demo script and rehearsal; seed polish; hosted instance frozen 48 h before; laptop demo with tunnel fallback; README checklist; backup video | M3 | Runnable app, hosted URL, README, demo script, report | 2 / 3 | Dry run under 10 minutes covering journeys A–E and onboarding; one-command local start | Last-minute regressions |
| M5 Optional (if ahead) | BERTopic themes; Slack bot; real-time SSE; CSAT | M4 | Stretch items behind feature flags | as availabl e | Each independently switchable | None critical |

## Module ownership

| Person | Owns | Provides | Consumes |
|---|---|---|---|
| P1: Core, Governance & Eval runner | policy/, clocks/, geb/, workflows/, skills/, db/, RBAC, gateway/, eval/runner, eval/metrics, developer guide, licence audit | effect() contract, evaluate(), skill decorator, schema, OpenAPI, complete()/classify() interface, event stream for eval | Adapter interface, seed data, scenarios |
| P2: Service lines, UI, Data & Delivery | cases/, adapters/, ui/, definitions/*, seed generators, nlu/ (SetFit training + calibration), eval/scenarios, eval/baselines/b0, eval/reports, README, demo script, hosting | Adapter interface, definitions, seeds, labelled intents, scenarios, reports | OpenAPI, effect() contract, event names |

**Shared:** B1 baseline (P1 writes the agent loop in about a day; P2 wires its scenarios). Interfaces frozen at M0: EffectRequest/EffectResult, adapter interface, event names; OpenAPI frozen at the end of M1 week 1.

**Cut list if we slip (in order):** BERTopic → chat beyond Q&A → scenario count to 20 → DPDP request case type → second drill → Insights to a static view.

## J. Trade-offs and scaling plan

**Versus the simpler alternative (Approach A without the harness):** about 30–40% more engineering for evidence. The GEB, rules and runner are identical; the harness reuses the same adapters and events, so the marginal cost is scenario authoring and report code. The harness degrades gracefully (fewer scenarios) without touching the demo.

**Versus the more complex alternative (Approach C):** Temporal, OPA, a model proxy and a stream bus would each add an operational component and a learning curve for two people. The Postgres queue gives at-least-once delivery and the GEB ledger gives effect-level idempotency, which together produce the property the brief tests. OPA's benefits are approximated by versioned YAML rule sets with tests and a schema.

**Model-size trade-off:** a 1.7–4B model limits chat fluency, but the architecture places no decision authority in the LLM; intent and routing come from SetFit plus rules, and the LLM only emits schema-constrained typed outputs. The measured baseline B1 runs on the same model, so the comparison is fair; a large-model baseline (B2) is reported as a labelled list-price projection, not a measurement.

| Bottleneck | Symptom | Scaling path |
|---|---|---|
| Postgres queue under high step volume | Lock contention, latency | Partition by workflow; move runner to Temporal (same effect() contract) |
| Single llama-server | Chat latency under concurrent users | vLLM with continuous batching; or hosted Apache-2.0 model via config |
| SetFit in-process | CPU spikes on bursts | Separate inference worker; batch classification |
| Streaming export | Long-running connections | Async export jobs to object storage; signed download links |
| Audit table growth | Slow queries | Monthly partitions; archive to cold storage per retention policy |
| Rule evaluation | Many rules per decision | Compile rule sets to decision tables; cache by version |
| Single tenant | TBO group companies | Tenant column plus row-level security; definitions namespaced per tenant |
| Free-tier hosting | Cold starts; connection caps | Paid tier or self-hosted VM when volumes justify it |

**Honest limitation:** the demo proves properties on synthetic data and mock APIs; near-real-time sync at scale is a documented path, not a demonstrated capability.

## K. Updated Research Ledger (v6.1)

| Item | Status |
|---|---|
| Confirmed context | Team of 2; consumer GPU [A: 4–6 GB VRAM]; free-tier hosting for the always-on demo |
| Architecture | Single service (helm-core) + Postgres + React + local llama-server + helm-eval; GEB as sole side-effect path; definitions-driven service lines |
| Verified this phase | llama-server OpenAI-compatible endpoints, JSON mode and json_schema grammar; chat and embedding models cannot share one process |

| Item | Status |
|---|---|
| Model tiering | 4 GB → Qwen3-1.7B or Gemma 4 E2B; 6 GB → Qwen3-4B; ≥8 GB → optional Qwen3-8B comparator; model cards and licences to verify (Tier 3) |
| Baselines | B0 and B1 measured; B2 dropped as a measurement, replaced by a labelled list-price projection |
| Hosting | Hosted CPU-only instance with MODEL_PROVIDER=none; laptop instance for LLM features; optional tunnel bridge via config |
| Stack decisions | FastAPI, SQLAlchemy/Alembic, PostgreSQL 16, React/Vite/Tailwind, llama.cpp, small Apache-2.0 GGUF model, SetFit, scikit-learn, Presidio, PyYAML/jsonschema, workalendar, httpx, pytest/Hypothesis, Docker Compose; LangGraph for B1 only |
| Deferred components | Temporal, OPA/Cedar, LiteLLM proxy, vLLM, Redis/NATS, OpenTelemetry, multi-tenancy, BERTopic |
| Team allocation | P1 core/governance/gateway/eval-runner; P2 cases/adapters/UI/definitions/NLU/scenarios/reports/delivery |
| Effort | ~56 pd total, 28 each; cut list defined |
| Interfaces frozen at M0 | EffectRequest/EffectResult; adapter interface; event names |
| Not verified | Licences: llama.cpp, SetFit, sentence-transformer body, workalendar, Faker; Gemma 4 model-card licence; TBO mock API contract (U1); exact VRAM; free-tier limits (storage, connections, sleep behaviour) |
| Next step | Detailed evaluation protocol (scenario catalogue, metric definitions, cost method), then M0 |